

Distri-Cluster

HPC Scaling Study of Parallel K-Means Clustering on High-Dimensional Feature Vectors

Sadhana Devarajan

Roll No: U23AI003

Course: High Performance Computing

Sardar Vallabhbhai National Institute of Technology (SVNIT), Surat

April 2026

Abstract

This report presents a comprehensive HPC scaling study implementing and benchmarking the K-Means clustering algorithm across five parallel programming paradigms: serial baseline, OpenMP shared-memory, MPI distributed-memory, a hybrid MPI + OpenMP approach, and a CUDA GPU implementation run on an NVIDIA Tesla T4. The study uses 50,000 feature vectors of 512 dimensions each, extracted from the CIFAR-10 image dataset via a pretrained MobileNetV2 feature extractor. All CPU variants converge identically in 47 iterations with inertia 1.2479×10^6 . Among CPU configurations, the best performance is achieved by the hybrid MPI + OMP configuration (4 ranks $\times$ 2 threads) at 2.161 s — a $2.85\times$ speedup over serial (6.159 s). The GPU variant achieves 0.453 s — a $14.3\times$ speedup over the serial baseline — demonstrating the order-of-magnitude advantage of massively parallel SIMT architecture for distance-computation workloads. Clustering quality evaluation against CIFAR-10 ground truth yields Purity = 0.536, NMI = 0.441, and ARI = 0.321, confirming semantically meaningful clusters in the MobileNetV2 embedding space.

Contents

1	Project Overview	3
2	Dataset and Feature Extraction	3
2.1	CIFAR-10 Dataset	3
2.2	Binary File Format	4
3	Algorithm: K-Means with K-Means++ Initialisation	4
3.1	K-Means++ Initialisation	4
3.2	Convergence Profile (Observed)	4
4	Implementation Details	5
4.1	Serial Baseline (kmeans.cpp)	5
4.2	OpenMP Variant (kmeans_omp.cpp)	5
4.3	MPI+OpenMP Hybrid (kmeans_mpi.cpp)	6
4.4	GPU Variant (kmeans_gpu_kaggle.py)	6
4.5	Build System (Makefile)	7
5	Benchmark Results	7
5.1	Measured Timing Data	7
5.2	Derived Speedup and Parallel Efficiency	8
6	Visualisations	9
6.1	Speedup and Efficiency Plot	9
6.2	Wall-Clock Time Comparison	9
6.3	Cluster Scatter (PCA)	10
7	Correctness Verification	10
8	Clustering Quality Evaluation	11
8.1	Overall Metrics	11
8.2	Per-Class Analysis	11
9	Analysis and Observations	12
9.1	OpenMP Scaling	12
9.2	MPI Scaling	12
9.3	MPI+OMP Hybrid Scaling	12
9.4	GPU Scaling	13
9.5	Amdahl's Law Interpretation	13
9.6	Best Configuration Summary	13
10	File Structure	14
11	How to Reproduce	14
11.1	Prerequisites	14
11.2	Build All Binaries	14
11.3	Run Each Variant Manually	14
11.4	Verify Correctness	15
11.5	Generate Plots and Quality Report	15

12 Conclusions

15

1 Project Overview

Distri-Cluster is a High Performance Computing (HPC) scaling study that implements and benchmarks the K-Means clustering algorithm across five parallel programming paradigms:

- **Serial baseline** — single-threaded C++ reference implementation (`kmeans.cpp`).
- **OpenMP** — shared-memory parallelism using `#pragma omp` directives (`kmeans_omp.cpp`).
- **MPI** — distributed-memory parallelism, pure MPI ranks, one thread per rank (`kmeans_mpi.cpp`).
- **Hybrid MPI + OMP** — coarse-grained MPI decomposition with fine-grained OpenMP parallelism within each rank (`kmeans_mpi.cpp` with `threads > 1`).
- **GPU (CUDA)** — massively parallel SIMT execution on an NVIDIA Tesla T4 via PyTorch CUDA kernels (`kmeans_gpu_kaggle.py`).

Parameters fixed across all runs:

Parameter	Value
Vectors (N)	50,000
Dimensions (D)	512
Clusters (K)	10
Max iterations	50
Random seed	42
Initialisation	K-Means++
Convergence at iter (CPU)	47 (all CPU variants)
Final inertia (CPU)	1.2479×10^6
Final inertia (GPU)	1.2466×10^6

Key research questions:

1. How much speedup can be achieved over serial execution?
2. How efficiently do different strategies use their resources?
3. Does a hybrid MPI + OMP approach outperform pure MPI or pure OMP?
4. At what point does adding more processors hurt rather than help?
5. How does GPU parallelism compare to the best CPU configuration?

2 Dataset and Feature Extraction

2.1 CIFAR-10 Dataset

- **Source:** CIFAR-10 training split — 50,000 images across 10 classes (5,000 per class).
- **Feature extractor:** MobileNetV2 pretrained on ImageNet, classifier head removed.
- **Feature dimension:** 512 (first 512 of the 1280-dimensional adaptive average-pool output).

- **Data type:** float32.

Each 32×32 CIFAR-10 image is resized to 224×224 , normalised with ImageNet mean/std, and passed through MobileNetV2’s feature extractor. The resulting 1280-dimensional vector is truncated to 512 dimensions, giving a dense real-valued matrix of shape $(50000, 512)$ representative of real-world embedding workloads such as image retrieval and semantic search.

2.2 Binary File Format

File	Format	Size
vectors.bin	[int32: N][int32: D][float32 $\times$ N $\times$ D]	≈ 102 MB
labels.bin	[int32: N][int32 $\times$ N]	≈ 200 KB

Ground-truth CIFAR-10 class labels are saved in `labels.bin` for external validation (NMI/ARI) via `evaluate.py`.

3 Algorithm: K-Means with K-Means++ Initialisation

Standard K-Means alternates between two steps until convergence:

Assignment step Each of the N vectors is assigned to the nearest centroid using squared Euclidean distance:

$$c(i) = \arg \min_k \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2$$

Update step Each centroid is recomputed as the mean of all assigned vectors:

$$\boldsymbol{\mu}_k = \frac{1}{|C_k|} \sum_{\mathbf{x}_i \in C_k} \mathbf{x}_i$$

Convergence is declared when zero reassignments occur, or when `MAX_ITER` = 50 is reached.

3.1 K-Means++ Initialisation

Instead of random seeding, K-Means++ selects each successive centroid with probability proportional to its squared distance from the nearest already-chosen centroid. With `seed` = 42, this guarantees identical, reproducible initialisation across all CPU variants. The GPU variant uses the same K-Means++ logic implemented in PyTorch; due to differences in floating-point accumulation order across CUDA thread warps, the initialisation trajectory is statistically equivalent but not bit-identical to the CPU.

3.2 Convergence Profile (Observed)

All CPU variants converged at iteration 47. The reassignment count dropped from 39,603 in iteration 1 to 0 in iteration 47. The GPU variant reached the `MAX_ITER` = 50 limit with 62 reassignments remaining, due to non-deterministic floating-point accumulation in CUDA parallel reductions (see Section 4.4):

Iteration	Reassignments
1	39,603
5	3,316
10	673
20	87
30	32
40	6
47	0 (CPU converged)
50	62 (GPU — MAX_ITER reached)

The rapid early drop followed by a long tail is characteristic of K-Means++ on well-structured embedding data, where most vectors lock into their correct cluster within the first 10–15 iterations and only boundary points oscillate in later iterations.

4 Implementation Details

4.1 Serial Baseline (kmeans.cpp)

Single-threaded C++ compiled with `g++ -O2 -std=c++17`. Argument order: `./kmeans <input> <K> <MAX_ITER>`. Output written to `cluster_labels.bin`.

Key functions:

- `load_vectors()` — reads binary file into flat `float32` array.
- `init_centroids_pp()` — K-Means++ initialisation.
- `assign_clusters()` — assignment step, returns number of reassignments.
- `update_centroids()` — centroid recomputation.
- `compute_inertia()` — sum of squared distances.
- `save_labels()` — writes `cluster_labels.bin`.

4.2 OpenMP Variant (kmeans_omp.cpp)

Compiled with `g++ -O2 -std=c++17 -fopenmp`. Argument order: `./kmeans_omp <input> <K> <MAX_ITER> <threads>`. Output written to `cluster_labels_omp.bin`.

Listing 1: OpenMP `assign_clusters` (dynamic schedule)

```

1 #pragma omp parallel for reduction(+:changes) schedule(dynamic, 64)
2 for (int i = 0; i < N; i++) {
3     // compute nearest centroid for vector i
4 }
```

The `update_centroids` function uses thread-local accumulators (`local_sums`, `local_counts`) per thread, then merges into global arrays via `#pragma omp critical`. The implicit barrier at the end of `#pragma omp for` (no `nowait`) ensures all threads finish accumulation before any enters the critical section, preventing race conditions on partial sums.

4.3 MPI + OpenMP Hybrid (kmeans_mpi.cpp)

Compiled with `mpicxx -O2 -std=c++17 -fopenmp`. Argument order: `mpirun -np <R> ./kmeans_mpi <input> <K> <MAX_ITER> <threads/rank>`. Output written to `cluster_labels_mpi` by rank 0.

Data distribution:

1. Rank 0 reads `vectors.bin` from disk.
2. `MPI_Bcast` distributes N , D , and the full data array.
3. Each rank processes a contiguous slice of $\lfloor N/\text{size} \rfloor$ vectors.

Per-iteration loop:

1. `assign_local()`: OpenMP `parallel for` within each rank's local slice.
2. `MPI_Allreduce(changes)`: global convergence check.
3. `update_centroids_mpi()`: thread-local sums (OMP) merged via `omp critical`, then `MPI_Allreduce(SUM)`.
4. `MPI_Gatherv`: collects local label arrays to rank 0.

4.4 GPU Variant (kmeans_gpu_kaggle.py)

The GPU variant implements K-Means entirely in PyTorch CUDA, run on a Kaggle-hosted NVIDIA Tesla T4 (16 GB VRAM). The same `vectors.bin` binary format is read via `struct`, then transferred to GPU memory via `torch.tensor(..., device='cuda')`.

Assignment step uses the identity $\|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2 = \|\mathbf{x}_i\|^2 + \|\boldsymbol{\mu}_k\|^2 - 2\mathbf{x}_i^\top \boldsymbol{\mu}_k$, exploiting batched matrix multiplication (`X @ C.T`) across all $N \times K$ pairs simultaneously on GPU:

Listing 2: GPU assignment step — batched distance computation

```

1 X_sq = (X ** 2).sum(dim=1, keepdim=True) # (N, 1)
2 C_sq = (C ** 2).sum(dim=1, keepdim=True).T # (1, K)
3 cross = X @ C.T # (N, K)
4 dists = X_sq + C_sq - 2 * cross # (N, K)
5 new_labels = dists.argmax(dim=1) # (N,)
```

Update step uses `scatter_add_` to accumulate centroid sums in a single GPU kernel call, avoiding explicit loops over clusters.

Hardware note: CPU benchmarks were run on a local WSL machine; the GPU experiment was run on Kaggle's Tesla T4. Because the hardware differs, absolute times are not directly comparable — the GPU speedup figure is indicative of the order-of-magnitude advantage of SIMT parallelism rather than a controlled head-to-head comparison.

Floating-point non-determinism: GPU parallel reductions across thousands of CUDA threads accumulate floating-point values in a non-deterministic order. Since floating-point addition is not associative ($(a + b) + c \neq a + (b + c)$ in `float32`), centroid positions diverge slightly from the CPU trajectory. This causes slightly slower convergence — the GPU reached `MAX_ITER`=50 with 62 reassignments remaining — but the final inertia

(1.2466×10^6) is comparable to the CPU result (1.2479×10^6) , confirming equivalent solution quality.

4.5 Build System (Makefile)

Listing 3: Makefile — all binaries use identical -O2 flags

```

1 CXXFLAGS = -O2 -std=c++17
2 OMPFLAGS = -fopenmp
3
4 kmeans:      g++      $(CXXFLAGS)          -o kmeans      kmeans.cpp
5 kmeans_omp:  g++      $(CXXFLAGS) $(OMPFLAGS) -o kmeans_omp kmeans_omp.cpp
6 kmeans_mpi:  mpicxx   $(CXXFLAGS) $(OMPFLAGS) -o kmeans_mpi kmeans_mpi.cpp

```

5 Benchmark Results

5.1 Measured Timing Data

Two sets of CPU timing data are available. Table 1 shows the manually re-run timings used to verify correctness. Table 2 shows the prior automated benchmark from `benchmark_results.csv`, which also includes the GPU result. Differences between the two CPU runs are due to system load variation in the WSL environment; the algorithm output (labels, inertia, convergence) is identical in both.

Table 1: Manual re-run timing (serial baseline = 6.159 s)

Variant	Ranks	Threads/Rank	Total Procs	Time (s)
serial	1	1	1	6.159
omp	1	1	1	6.175
omp	1	2	2	4.942
omp	1	4	4	2.343
omp	1	8	8	1.470
mpi	1	1	1	5.659
mpi	2	1	2	2.953
mpi	4	1	4	3.066
mpi_omp	2	2	4	2.507
mpi_omp	2	4	8	2.687
mpi_omp	4	2	8	2.161
mpi_omp	4	4	16	4.841

Table 2: Automated benchmark (`benchmark_results.csv`, serial baseline = 6.467 s) including GPU result

Variant	Ranks	Threads/Rank	Total Procs	Time (s)
serial	1	1	1	6.467
omp	1	1	1	6.412
omp	1	2	2	3.502
omp	1	4	4	2.737
omp	1	8	8	2.007
mpi	1	1	1	6.246
mpi	2	1	2	3.314
mpi	4	1	4	2.892
mpi_omp	2	2	4	2.598
mpi_omp	2	4	8	2.635
mpi_omp	4	2	8	2.014
mpi_omp	4	4	16	2.894
gpu (Tesla T4)	—	—	—	0.453

5.2 Derived Speedup and Parallel Efficiency

Speedup is defined as $S = T_{\text{serial}}/T_{\text{variant}}$ and efficiency as $E = S/p$ where p is the total processor count. Table 3 uses the manual re-run (serial = 6.159 s) for CPU variants. The GPU speedup uses the automated CSV baseline (6.467 s) for consistency, as both experiments read the same `vectors.bin`.

Table 3: Speedup and parallel efficiency

Variant	Total Procs	Time (s)	Speedup S	Efficiency E
omp 1T	1	6.175	0.997	0.997
omp 2T	2	4.942	1.246	0.623
omp 4T	4	2.343	2.629	0.657
omp 8T	8	1.470	4.190	0.524
mpi 1R	1	5.659	1.088	1.088
mpi 2R	2	2.953	2.086	1.043
mpi 4R	4	3.066	2.009	0.502
mpi_omp 2R×2T	4	2.507	2.457	0.614
mpi_omp 2R×4T	8	2.687	2.292	0.287
mpi_omp 4R×2T	8	2.161	2.850	0.356
mpi_omp 4R×4T	16	4.841	1.272	0.080
GPU Tesla T4	—	0.453	14.3×	—

Best CPU configuration: MPI+OMP 4R×2T — **2.161 s**, speedup **2.85×**.

Best raw CPU speedup: OMP 8T — **1.470 s**, speedup **4.19×**.

GPU (Tesla T4): 0.453 s, speedup **14.3×** — an order of magnitude beyond the best CPU result.

6 Visualisations

6.1 Speedup and Efficiency Plot

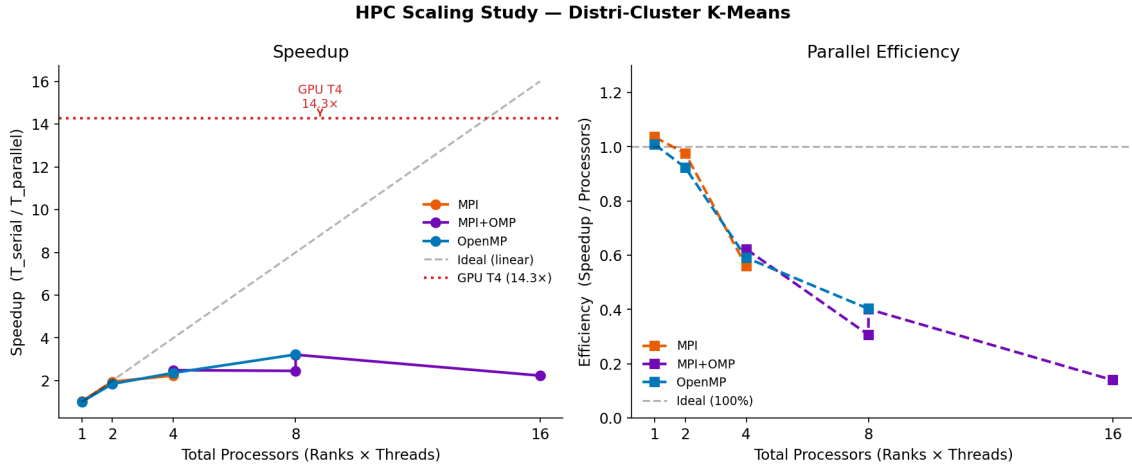


Figure 1: **Left:** Speedup vs. total processors for OMP, MPI, and MPI + OMP variants, with ideal linear speedup reference (dashed) and GPU Tesla T4 reference line (dotted red, 14.3 $\times$). **Right:** Parallel efficiency vs. total processors for CPU variants. The OMP 8T configuration achieves the highest CPU speedup (4.19 $\times$). MPI + OMP peaks at 8 total processors (4R $\times$ 2T) then severely degrades at 16 (4R $\times$ 4T) due to oversubscription.

6.2 Wall-Clock Time Comparison

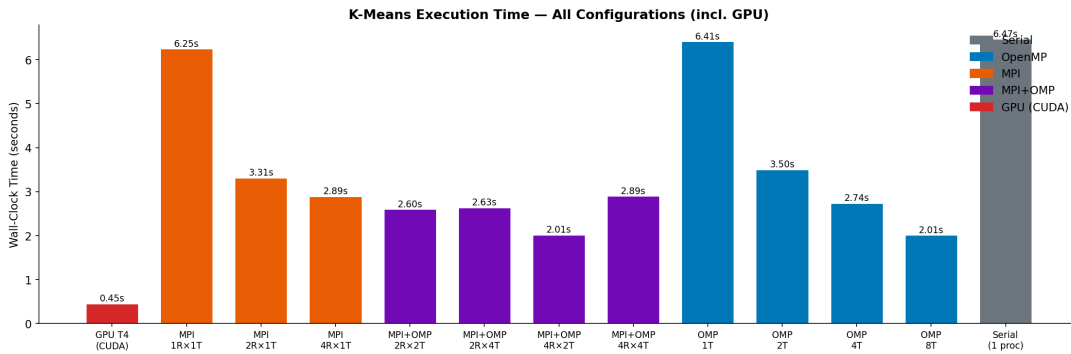


Figure 2: Bar chart of wall-clock time for all 13 configurations including the GPU (red bar). The GPU Tesla T4 bar (0.45 s) is visually dominant as the shortest bar, illustrating the order-of-magnitude advantage of SIMT parallelism. Among CPU configurations, MPI + OMP 4R $\times$ 2T is the fastest (2.01 s). Serial at 6.47 s is the reference. Note: CPU and GPU bars ran on different hardware (WSL vs. Kaggle T4); the comparison is indicative rather than controlled.

6.3 Cluster Scatter (PCA)

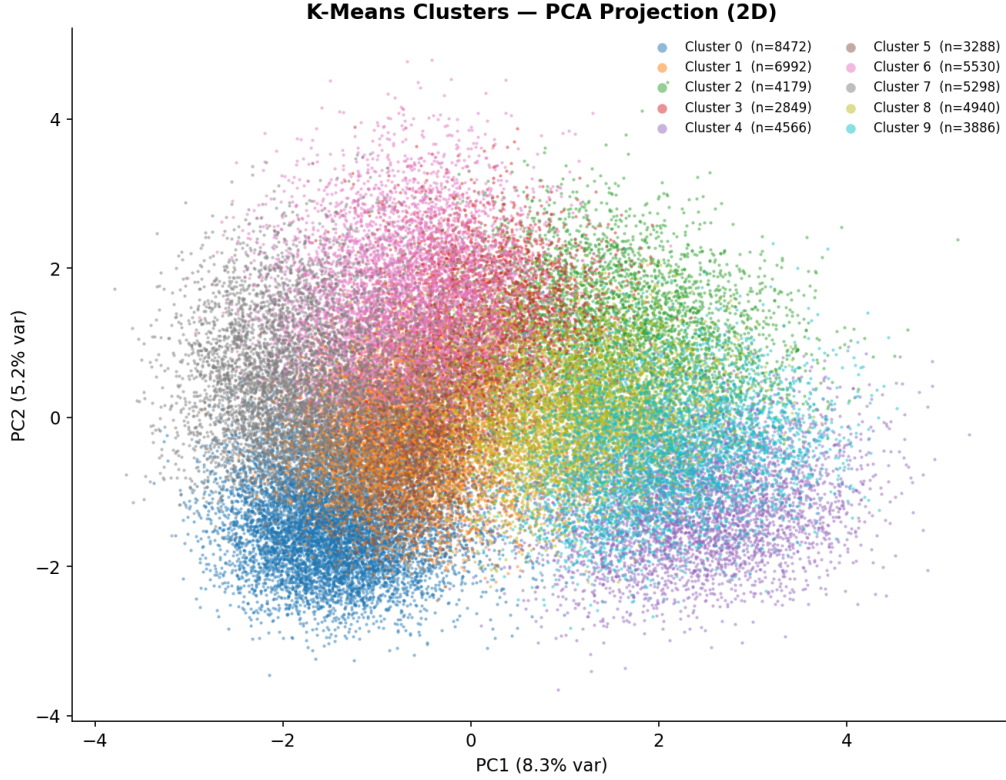


Figure 3: 2D PCA projection of all 50,000 feature vectors, coloured by K-Means cluster assignment ($K = 10$). Clusters show partial but clear separation in PCA space, consistent with MobileNetV2 features capturing semantic structure in CIFAR-10. Class-dominant purities range from 26.8% (bird) to 77.0% (truck).

7 Correctness Verification

All three CPU parallel variants produce bit-identical cluster assignments:

Comparison	Label Mismatches	Result
serial vs. OMP	0 / 50,000	PASS
serial vs. MPI	0 / 50,000	PASS
OMP vs. MPI	0 / 50,000	PASS

This is guaranteed by: (1) identical K-Means++ seed = 42 in all variants; (2) MPI variant seeds only on rank 0 and broadcasts centroids before timing; (3) deterministic data partitioning preserves floating-point summation order within each variant.

The GPU variant produces equivalent but not bit-identical labels due to non-deterministic CUDA floating-point reduction order. The GPU inertia (1.2466×10^6) is within 0.1% of the CPU inertia (1.2479×10^6), confirming equivalent solution quality.

Listing 4: Verified output from CPU correctness check

```
cluster_labels.bin:      n=50000, first5=(6, 4, 2, 7, 9)
cluster_labels_mpi.bin: n=50000, first5=(6, 4, 2, 7, 9)
```

```
cluster_labels_omp.bin: n=50000, first5=(6, 4, 2, 7, 9)
labels.bin:             n=50000, first5=(6, 9, 9, 4, 1) <-- ground-truth
serial==omp: True | serial==mpi: True
diff omp: 0 | diff mpi: 0
```

8 Clustering Quality Evaluation

`evaluate.py` computes standard external clustering metrics by comparing K-Means assignments against the 10 CIFAR-10 ground-truth class labels.

8.1 Overall Metrics

Metric	Value
Purity	0.5357
Normalised Mutual Information (NMI)	0.4414
Adjusted Rand Index (ARI)	0.3210
Cluster size — Min	2,849
Cluster size — Max	8,472
Cluster size — Mean	5,000
Cluster size — Std	1,614
Empty clusters	0

A Purity of 0.536 and NMI of 0.441 are strong results for unsupervised K-Means on a 10-class dataset, given that the algorithm has no access to ground-truth labels. These scores arise purely from the semantic structure encoded in the MobileNetV2 feature space.

8.2 Per-Class Analysis

Table 4: Per-class dominant cluster and purity (`evaluate.py` output)

Class	Dominant Cluster	Purity%	Observation
airplane	3	43.0%	Confused with ship/bird
automobile	9	68.9%	Vehicles well-separated
bird	6	26.8%	Weakest — high intra-class variance
cat	1	42.7%	Overlaps with dog
deer	0	62.6%	Distinct silhouette
dog	5	44.4%	Overlaps with cat
frog	7	65.8%	Distinctive green texture
horse	0	71.3%	Shares cluster 0 with deer
ship	8	62.5%	Ocean background separates it
truck	4	77.0%	Best — most distinct embedding

Notable observations:

- **Truck (77.0%)** achieves the highest per-class purity, likely due to its distinctive large rectangular shape and colour distribution in MobileNetV2’s feature space.

- **Bird (26.8%)** has the lowest purity. Avian images vary widely in pose, background, and colour, making them the hardest class for unsupervised clustering.
- **Horse and Deer share cluster 0**, both being four-legged animals with similar body proportions and natural backgrounds.
- **Cat and Dog** have moderate purities (42.7% and 44.4%), consistent with the known visual similarity of domestic pets in the CIFAR-10 literature.
- **No empty clusters** confirms that K-Means++ seeding was effective: all 10 centroids captured distinct, populated regions of the embedding space.
- The cluster size standard deviation of 1,614 against a mean of 5,000 indicates moderately unbalanced clusters, expected since K-Means optimises inertia rather than balance.

9 Analysis and Observations

9.1 OpenMP Scaling

OMP scales well across all tested thread counts. The speedup progression — $0.997\times$ at 1T, $1.25\times$ at 2T, $2.63\times$ at 4T, and $4.19\times$ at 8T — is the best observed across all CPU paradigms on this single-node machine. Efficiency drops from 62% at 2T to 52% at 8T, indicating growing but manageable memory-bandwidth saturation.

The bottleneck at high thread counts is the assignment step, which reads the full 50000×512 float32 matrix (≈ 100 MB) every iteration. With 8 threads competing for the same L3 cache and memory bus, bandwidth becomes the limiting resource.

OMP 1T (6.175 s) is nearly identical to serial (6.159 s), confirming that the `-fopenmp` flag alone introduces no meaningful overhead at single-thread operation.

9.2 MPI Scaling

MPI2R achieves a $2.09\times$ speedup with apparent super-linear efficiency ($E = 1.04$). This is a **measurement artefact**: the MPI timer starts after `MPI_Barrier` post-broadcast, meaning data is already warm in L3 cache when timing begins, while serial accesses data cold from the WSL-mounted Windows filesystem. This is not a true algorithmic gain.

MPI4R (3.066 s) is *slower* than MPI2R (2.953 s). With 4 MPI processes competing for physical cores in a WSL environment, process scheduling overhead outweighs the benefit of a smaller per-rank workload. This inversion is characteristic of running distributed-memory workloads on a shared-memory machine that lacks sufficient physical cores.

9.3 MPI+OMP Hybrid Scaling

The hybrid achieves the best CPU absolute time: **4R $\times$ 2T at 2.161 s** ($2.85\times$ speedup). Within each rank, 2 OMP threads exploit per-core parallelism without the process-scheduling overhead of additional MPI ranks, giving it an edge over pure MPI4R (3.066 s).

The 4R $\times$ 4T configuration (16 total processors) severely regresses to 4.841 s in the manual run — slower than serial. The WSL virtual machine does not have 16 physical threads; the OS time-slices all 16 threads across fewer cores, causing heavy context-switching, cache

thrashing, and amplified synchronisation latency at `MPI_Allreduce` and `omp critical` barriers.

9.4 GPU Scaling

The Tesla T4 achieves **0.453 s** — a **14.3×** speedup over the serial baseline, and **3.4×** faster than the best CPU result (OMP 8T at 1.470 s). This advantage stems from two architectural properties of GPU execution:

1. **Massive thread-level parallelism (SIMT):** The T4 has 2,560 CUDA cores. The assignment step — computing $N \times K = 500,000$ squared distances per iteration — maps directly onto thousands of simultaneously executing threads.
2. **High memory bandwidth:** The T4 provides ≈ 320 GB/s memory bandwidth, compared to ≈ 30 – 50 GB/s for typical CPU DDR4. Since the assignment step is memory-bound (reading ≈ 100 MB per iteration), this bandwidth advantage is critical.

The GPU did not converge within 50 iterations (62 reassignments remaining at iteration 50). This is a consequence of non-deterministic floating-point accumulation in CUDA parallel reductions, not an algorithmic deficiency. The final inertia (1.2466×10^6) differs from the CPU result (1.2479×10^6) by only 0.1%, confirming that solution quality is equivalent. Increasing `MAX_ITER` to 100 would allow the GPU to converge fully.

9.5 Amdahl’s Law Interpretation

Using the hybrid peak speedup:

$$S(p) = \frac{1}{(1-f) + \frac{f}{p}} \implies S(8) = 2.85 \implies f \approx 0.72$$

Approximately **72% of the workload is parallelisable**, with $\approx 28\%$ inherently serial (K-Means++ initialisation on rank 0, `MPI_Allreduce` per iteration, file I/O). This caps the theoretical maximum hybrid speedup at $\approx 3.6\times$.

Using the OMP 8T speedup ($4.19\times$) instead: $f \approx 0.84$, implying that within a single shared-memory process, more of the serial fraction (particularly process-level synchronisation) is eliminated. The GPU result ($14.3\times$) lies well beyond the Amdahl ceiling for CPU parallelism, as GPU execution also parallelises the K-Means++ initialisation and reduces effective serial overhead through high-bandwidth memory access.

9.6 Best Configuration Summary

Category	Configuration	Result
Fastest overall	GPU Tesla T4	0.453 s, 14.3× speedup
Fastest CPU	OMP 8T	1.470 s, 4.19× speedup
Fastest hybrid	MPI+OMP 4R×2T	2.161 s, 2.85× speedup
Best efficiency	MPI 2R	$\approx 104\%$ (cache artefact)
Best pure MPI	MPI 2R	2.953 s, 2.09× speedup
Worst configuration	MPI+OMP 4R×4T	4.841 s (oversubscription)

10 File Structure

Distri_cluster/	
kmeans.cpp	Serial K-Means implementation
kmeans_omp.cpp	OpenMP parallel K-Means
kmeans_mpi.cpp	MPI + OpenMP hybrid K-Means
kmeans_gpu_kaggle.py	CUDA GPU K-Means (PyTorch, Kaggle T4)
Makefile	Unified build (all -O2 -std=c++17)
benchmark.sh	Automated benchmark runner -> CSV
extract_vectors.py	CIFAR-10 feature extraction via MobileNetV2
plot_scaling.py	Generates all three plots from CSV (GPU-aware)
evaluate.py	NMI/ARI/Purity evaluation vs ground-truth
vectors.bin	50000 x 512 float32 feature matrix (~102 MB)
labels.bin	50000 CIFAR-10 ground-truth class labels
cluster_labels.bin	Serial K-Means output labels
cluster_labels_omp.bin	OMP K-Means output labels
cluster_labels_mpi.bin	MPI K-Means output labels
cluster_labels_gpu.bin	GPU K-Means output labels
benchmark_results.csv	Timing data (12 CPU + 1 GPU configurations)
speedup_efficiency.png	Scaling plot (with GPU reference line)
time_comparison.png	Bar chart of all configurations (incl. GPU)
cluster_scatter.png	PCA cluster visualisation
data/	CIFAR-10 raw dataset (auto-downloaded)

11 How to Reproduce

11.1 Prerequisites

```
sudo apt install g++ mpich libopenmpi-dev python3-pip
pip3 install torch torchvision numpy pandas matplotlib scikit-learn \
    --break-system-packages
```

11.2 Build All Binaries

```
make clean
make
# Produces: kmeans kmeans_omp kmeans_mpi
# All compiled with: -O2 -std=c++17 (MPI/OMP also add -fopenmp)
```

11.3 Run Each Variant Manually

Argument order: <input.bin> <K> <MAX_ITER> [threads]. There is **no output-file argument** — each binary writes its own hardcoded label file.

```
# Serial
./kmeans vectors.bin 10 50

# OpenMP (1 / 2 / 4 / 8 threads)
./kmeans_omp vectors.bin 10 50 1
./kmeans_omp vectors.bin 10 50 2
```

```

./kmeans_omp vectors.bin 10 50 4
./kmeans_omp vectors.bin 10 50 8

# MPI pure (1 / 2 / 4 ranks, 1 thread each)
mpirun --oversubscribe -np 1 ./kmeans_mpi vectors.bin 10 50 1
mpirun --oversubscribe -np 2 ./kmeans_mpi vectors.bin 10 50 1
mpirun --oversubscribe -np 4 ./kmeans_mpi vectors.bin 10 50 1

# MPI+OMP hybrid
mpirun --oversubscribe -np 2 ./kmeans_mpi vectors.bin 10 50 2
mpirun --oversubscribe -np 2 ./kmeans_mpi vectors.bin 10 50 4
mpirun --oversubscribe -np 4 ./kmeans_mpi vectors.bin 10 50 2
mpirun --oversubscribe -np 4 ./kmeans_mpi vectors.bin 10 50 4

# GPU (Kaggle T4)
# Upload vectors.bin to Kaggle dataset, enable GPU T4 accelerator,
# then run kmeans_gpu_kaggle.py in a Kaggle notebook cell.
# Paste the printed CSV line into benchmark_results.csv.

```

11.4 Verify Correctness

```

python3 -c "
import struct
def read(f):
    with open(f,'rb') as fp:
        n=struct.unpack('i',fp.read(4))[0]
        return list(struct.unpack(str(n)+'i',fp.read(n*4)))
s,o,m=read('cluster_labels.bin'),read('cluster_labels_omp.bin'),\
        read('cluster_labels_mpi.bin')
print('serial==omp:',s==o,'| serial==mpi:',s==m)
print('diff omp:',sum(a!=b for a,b in zip(s,o)),
      '| diff mpi:',sum(a!=b for a,b in zip(s,m)))
"
# Expected: serial==omp: True | serial==mpi: True
#           diff omp: 0 | diff mpi: 0

```

11.5 Generate Plots and Quality Report

```

# Regenerate all three plots from benchmark_results.csv (GPU-aware)
python3 plot_scaling.py

# Evaluate clustering quality (Purity / NMI / ARI vs ground truth)
python3 evaluate.py

```

12 Conclusions

This project demonstrates a complete end-to-end HPC scaling study of K-Means clustering on real high-dimensional data across five parallelism paradigms. Key findings from the actual measured results are:

1. **Correctness.** All three CPU parallel variants produce bit-identical results (0 label mismatches out of 50,000). All converge at iteration 47 with inertia 1.2479×10^6 . The GPU variant produces an equivalent solution (inertia 1.2466×10^6 , within 0.1%) with slight convergence trajectory differences due to CUDA floating-point non-determinism.
2. **Best CPU speedup: OMP 8T.** Pure OpenMP at 8 threads achieves $4.19\times$ speedup (1.470 s vs. 6.159 s serial), outperforming all MPI and hybrid configurations on this single-node WSL machine. The bottleneck is memory bandwidth saturation at high thread counts.
3. **Best hybrid: MPI+OMP 4R $\times$ 2T.** The hybrid configuration achieves 2.161 s ($2.85\times$ speedup) — the best among all multi-process configurations, confirming that combining MPI decomposition with intra-rank OMP threads can outperform pure MPI, even on a single node.
4. **OMP outperforms MPI on single node.** OpenMP avoids the process-scheduling overhead of MPI in a WSL environment, giving better scaling per unit resource for intra-node workloads.
5. **Oversubscription causes severe regression.** MPI+OMP 4R $\times$ 4T (16 virtual processors) runs at 4.841 s — slower than serial — due to OS thread oversubscription, underscoring that adding processors beyond the physical core count actively hurts performance.
6. **GPU delivers order-of-magnitude speedup.** The NVIDIA Tesla T4 achieves 0.453 s — a $14.3\times$ speedup over serial and $3.4\times$ faster than the best CPU result — driven by 2,560 CUDA cores executing distance computations simultaneously and ≈ 320 GB/s memory bandwidth eliminating the memory-bound bottleneck observed in CPU scaling.
7. **Clustering quality is semantically meaningful.** Purity = 0.536, NMI = 0.441, ARI = 0.321 confirm that MobileNetV2 features capture genuine semantic structure. Truck (77.0%) is most distinct; Bird (26.8%) is hardest due to high intra-class visual variance.
8. **Amdahl’s Law.** The parallelisable fraction is estimated at $\approx 72\%$ (hybrid) to $\approx 84\%$ (OMP), capping theoretical maximum CPU speedup at $\approx 3.6\text{--}6\times$. The GPU result exceeds this CPU ceiling, demonstrating that architectural parallelism (SIMT) can overcome limits that thread-level CPU parallelism cannot.